P08 — Simulador de partículas con física básica

Lenguaje de Programación 1 (Imperativo)

Grupo 1

Tom Jordan Pérez López, Raul Antonio Guzmán López, Ricardo Gaspar Reyes Campos y Sandro Giancarlo Chavez Vicaña

Facultad de Ingeniería Industrial y de Sistemas, Universidad Nacional de Ingeniería

Ing. Juan Luis Herencia Guerra

Abril, 2026

Tabla de contenido

Resumen..... 3

Análisis y Diseño..... 4

Requerimientos Funcionales..... 4

Requerimientos no funcionales..... 6

Restricciones del Sistema..... 7

Descripción de las Ecuaciones Físicas..... 8

Integración de Movimiento (Método de Euler Semi-Implícito)..... 8

Gravedad (Aceleración Constante)..... 8

Fricción (Amortiguamiento Viscoso)..... 9

Colisión Elástica entre Partículas..... 9

Colisión con Paredes (Reflexión y Restitución)..... 11

Análisis de Energía del Sistema..... 12

Diseño de las Estructuras y Algoritmos de Simulación..... 14

Variables Adicionales..... 14

Parámetros físicos del sistema..... 14

Parámetros numéricos de simulación..... 14

Parámetros del dominio espacial..... 15

Diseño del Struct de Partícula..... 15

Descripción de los Atributos..... 16

Bucle de Simulación..... 17

Flujo lógico por cada tick:.....17

Análisis algorítmico..... 21

Entrada interactiva del usuario (módulo entrada.c)..... 22

Archivos CSV..... 22

Visualizador en Python.....24

Las bibliotecas y su rol.....25

pandas: lectura y manipulación tabular.....25

numpy: operaciones vectoriales..... 25

matplotlib.animation..... 25

Pillow: animación GIF..... 26

Resultados - entregables visuales.....26

Diagramas.....28

Referencias.....31

Resumen

El presente proyecto describe el diseño e implementación de un simulador de colisiones de partículas en 2D, estructurado como una plataforma dual para la física computacional y la generación estructurada de datos.

A un nivel técnico, el núcleo de la lógica principal está desarrollado en C, aplicando el método de integración de Euler semi-implícita para actualizar cinemáticamente el estado de las partículas bajo los efectos de una gravedad configurable y amortiguamiento viscoso. El motor detecta y resuelve colisiones elásticas bidimensionales y rebotes contra los límites del espacio previamente definido, exportando de manera continua las trayectorias, la variación de la energía mecánica total y las métricas de rendimiento en archivos de texto plano (CSV). Estos registros son posteriormente leídos por un módulo independiente programado en Python. Mediante el uso de bibliotecas del ecosistema científico (pandas, numpy, matplotlib y Pillow), el sistema procesa los datos para renderizar animaciones GIF de las trayectorias, mapas de densidad espacial (heatmaps) y gráficas para auditar la conservación energética.

Más allá de la visualización, lo que buscamos en el proyecto es consolidar una verdadera "fábrica de datos físicos". Por un lado, se provee a las ciencias exactas un entorno empírico para validar leyes termodinámicas y mecánicas; por otro, abastece al campo de la Ciencia de Datos (Data Science) con bases masivas, dinámicas y limpias, ideales para aplicar pruebas estadísticas complejas, modelos de regresión y algoritmos predictivos sobre eventos físicos.

Palabras clave: partícula, colisión, tick, python, C.

Análisis y Diseño

Requerimientos Funcionales

Tabla 1. Requerimientos funcionales del sistema.

Requerimientos Funcionales		
1	Inicialización de partículas	El sistema debe inicializar N partículas con posiciones aleatorias dentro del espacio de simulación 2D, sin solapamiento inicial. Cada partícula tendrá: posición (x, y), velocidad (vx, vy), masa y radio configurables o generados aleatoriamente dentro de rangos definidos. Además, cada tick hace referencia a la unidad de paso de tiempo (por ejemplo, ns, ms, s, etc.).
2	Integración de movimiento (Euler)	El simulador debe actualizar la posición de cada partícula en cada tick mediante integración de Euler hacia adelante (forward Euler), usando el paso de tiempo dt como parámetro configurable.
3	Aplicación de gravedad	El simulador debe aplicar una aceleración gravitacional constante *g* (configurable, por defecto 9.81 m/s²) sobre el eje vertical (vy) de cada partícula en cada tick.
4	Aplicación de fricción	El simulador debe aplicar un coeficiente de amortiguamiento viscoso (fricción) sobre ambas componentes de velocidad en cada tick, reduciendo gradualmente la energía cinética del sistema.
5	Detección de	El simulador debe detectar colisiones entre todos los pares posibles

	colisiones entre partículas	de partículas usando comparación de distancias euclidianas versus la suma de sus radios. Se deben evaluar N·(N−1)/2 pares únicos por tick.
6	Resolución de colisiones elásticas en dos dimensiones	Al detectarse una colisión entre dos partículas, el sistema debe resolver la respuesta usando las fórmulas de colisión elástica 2D: descomposición en componentes normal y tangencial, transferencia de momento y corrección de solapamiento (overlap correction).
7	Colisión con paredes	El simulador debe detectar cuando una partícula alcanza o sobrepasa los límites del espacio (ancho × alto) y aplicar reflexión con coeficiente de restitución configurable (0 = inelástico, 1 = perfectamente elástico).
8	Cálculo de energía del sistema	El simulador debe calcular en cada tick la energía cinética total (Ek), la energía potencial gravitacional total (Ep) y la energía mecánica total (Et = Ek + Ep) de todas las partículas.
9	Exportación a CSV	El simulador debe generar tres archivos CSV al finalizar la simulación: ● trayectorias.csv — columnas: tick, particula_id, x, y, vx, vy ● energia.csv — columnas: tick, energia_cinetica, energia_potencial, energia_total

		• benchmark.csv — columnas: n_particulas, ticks, tiempo_segundos, particulas_por_segundo
10	Visualización en Python	Un script visualizar.py debe leer los CSV generados y producir: animación de trayectorias, gráfico de energía vs. tiempo, heatmap de densidad de posiciones y gráfico de rendimiento (benchmark).
11	Benchmarking de rendimiento	El simulador debe medir y reportar el tiempo de ejecución con clock(), calcular la tasa de procesamiento (partículas×ticks / segundos) y guardarlos en benchmark.csv.

Nota: Elaboración propia.

Requerimientos no funcionales

Tabla 2. Requerimientos no funcionales del sistema.

	Lenguaje y estándar	C puro, estándar C99/C11, compilado con gcc -Wall -Wextra -O2 -lm.
	Modularidad	El código debe separarse en al menos cuatro archivos: simulador.c, particula.h, fisicas.c, csv_export.c.
	Gestión de memoria	Toda memoria dinámica (malloc/calloc) debe validarse y liberarse con free(). Toda apertura de archivo debe validarse con fopen.

	Escalabilidad	El sistema debe funcionar correctamente con N desde 2 hasta al menos 1000 partículas.
	Reproducibilidad	La semilla aleatoria (srand) debe poder fijarse por parámetro para reproducir simulaciones.
	Build system	Debe incluirse un Makefile con los targets: all, clean, profile, optimized.
	Versionamiento	El proyecto debe versionarse con Git y publicarse en GitHub con historial de commits descriptivos.

Nota: Elaboración propia.

Restricciones del Sistema

- El espacio de simulación es un rectángulo 2D cerrado con dimensiones configurables (ancho × alto).
- La resistencia del medio (fricción o amortiguamiento viscoso), por defecto, será igual a cero.
- La gravedad actúa únicamente en el eje Y (hacia abajo, positivo).
- Las colisiones entre partículas son siempre elásticas (conservan momento y energía cinética).
- El paso de tiempo dt debe ser suficientemente pequeño para estabilidad numérica (dt < 0.01 s recomendado).

Descripción de las Ecuaciones Físicas

Integración de Movimiento (Método de Euler Semi-Implícito)

Para la actualización del estado de las partículas en cada diferencial de tiempo (dt), o tick, se emplea la integración de Euler Semi-Implícita (o Euler-Cromer). A diferencia del método de Euler hacia adelante, este enfoque actualiza primero la velocidad y luego utiliza esta *nueva* velocidad para calcular la posición, lo cual mejora drásticamente la estabilidad numérica y la conservación de la energía en simulaciones físicas.

Las leyes de Newton establecen que la variación de la velocidad depende de la aceleración, y la variación de la posición depende de la velocidad. La discretización para el algoritmo es la siguiente:

1. Actualización de velocidad:

$$vx(t + dt) = vx(t) + ax(t) \cdot dt$$

$$vy(t + dt) = vy(t) + ay(t) \cdot dt$$

2. Actualización de posición:

- $x(t + dt) = x(t) + vx(t + dt) \cdot dt$
- $y(t + dt) = y(t) + vy(t + dt) \cdot dt$

Gravedad (Aceleración Constante)

La gravedad se modela como una fuerza constante que actúa exclusivamente sobre el eje vertical. Asumiendo una convención de coordenadas orientada a la pantalla (donde el eje Y

crece hacia abajo), la gravedad "g" se aplica como un valor positivo que incrementa la velocidad en Y:

- Aceleración horizontal: $a_x = 0$
- Aceleración vertical: $a_y = g$ (donde $g \approx 9.81$ m/s², valor configurable).

En cada tick, la velocidad vertical se actualiza como:

- $v_y(t + dt) = v_y(t) + g \cdot dt$

Fricción (Amortiguamiento Viscoso)

La fricción modela la resistencia del medio (como el aire o un fluido) y produce una fuerza de arrastre proporcional y opuesta a la velocidad. A nivel cinemático, esto se traduce en una desaceleración exponencial gradual. En el código, se multiplica la velocidad por un factor de atenuación basado en el coeficiente "μ":

- $v_x(t + dt) = v_x(t) \cdot (1 - \mu \cdot dt)$
- $v_y(t + dt) = v_y(t) \cdot (1 - \mu \cdot dt)$

Invariante de consistencia: El coeficiente de fricción μ (medido en s^{-1}) debe cumplir que: $\mu < 1/dt$. Si el factor $(1 - \mu \cdot dt)$ se vuelve negativo, la velocidad invertiría su dirección bruscamente por un error numérico, lo cual es físicamente inválido.

Colisión Elástica entre Partículas

Una colisión elástica perfecta conserva tanto el momento lineal del sistema como la energía cinética total.

Paso 1: Vector normal de colisión (n)

Se calcula el vector unitario que va desde el centro de la partícula 1 (p1) al centro de la partícula 2 (p2):

- dx = p2.x - p1.x
- dy = p2.y - p1.y
- d = sqrt($dx^2 + dy^2$) ← Distancia entre centros
- nx = dx / d
- ny = dy / d

Paso 2: Velocidad relativa y filtro de aproximación

Se proyecta la diferencia de velocidades sobre el vector normal para obtener un escalar que indica cuán rápido se acercan:

- dvx = p1.vx - p2.vx
- dvy = p1.vy - p2.vy
- vrel_n = dvx · nx + dvy · ny

Condición crítica: Si vrel_n ≤ 0, las partículas se están alejando mutuamente o se mueven en paralelo. La colisión se ignora para prevenir resoluciones dobles.

Paso 3: Cálculo y aplicación del impulso de colisión (J)

Con masas m_1 y m_2, el impulso escalar bidireccional se define como:

- $J = (2 \cdot m_1 \cdot m_2 \cdot vrel_n) / (m_1 + m_2)$

Se aplica este impulso a las velocidades, respetando la Tercera Ley de Newton (acción y reacción):

- $p1.vx = p1.vx - (J / m_1) \cdot nx$
- $p1.vy = p1.vy - (J / m_1) \cdot ny$
- $p2.vx = p2.vx + (J / m_2) \cdot nx$
- $p2.vy = p2.vy + (J / m_2) \cdot ny$

Paso 4: Corrección de solapamiento ponderada por masa

Para evitar que las partículas queden intersecadas por la naturaleza discreta del tiempo "dt", deben separarse distribuyendo la distancia de superposición según la inercia de cada una (la masa menor se desplaza más):

- $solapamiento = (p1.radio + p2.radio) - d$
- $factor_1 = m_2 / (m_1 + m_2)$
- $factor_2 = m_1 / (m_1 + m_2)$
- $p1.x = p1.x - (solapamiento \cdot factor_1 \cdot nx)$
- $p1.y = p1.y - (solapamiento \cdot factor_1 \cdot ny)$
- $p2.x = p2.x + (solapamiento \cdot factor_2 \cdot nx)$
- $p2.y = p2.y + (solapamiento \cdot factor_2 \cdot ny)$

Colisión con Paredes (Reflexión y Restitución)

Los límites del espacio simulan muros inamovibles. El coeficiente de restitución "e" (donde $0 \leq e \leq 1$) define qué fracción de la velocidad se conserva tras el impacto.

Eje X (Paredes izquierda/derecha):

- Si ($x - radio \leq 0$):

$vx = |vx| \cdot e$ (Reposicionar a: $x = radio$)

- Si ($x + radio \geq ancho$):

$vx = -|vx| \cdot e$ (Reposicionar a: $x = ancho - radio$)

Eje Y (Paredes superior/inferior):

- Si ($y - radio \leq 0$):

$vy = |vy| \cdot e$ (Reposicionar a: $y = radio$)

- Si ($y + radio \geq alto$):

$vy = -|vy| \cdot e$ (Reposicionar a: $y = alto - radio$)

Análisis de Energía del Sistema

Para auditar la estabilidad de la simulación, se monitoriza la energía mecánica total del sistema, denotada por el símbolo de sumatoria (Σ) sobre todas las partículas (i).

Energía Cinética (Ek): Dependiente de la masa y la magnitud al cuadrado de la velocidad.

- $Ek_{total} = \Sigma \left[\frac{1}{2} \cdot m_i \cdot (vx_i^2 + vy_i^2) \right]$

Energía Potencial Gravitacional (Ep): Dado que la convención del renderizado define el eje Y creciente hacia abajo, la altura real respecto al "suelo" de la pantalla es ($alto - y$).

- $Ep_{total} = \Sigma \left[m_i \cdot g \cdot (alto - y_i) \right]$

Energía Mecánica Total (Et):

- $Et = Ek_total + Ep_total$

Principio de Conservación: Si el sistema se configura sin fricción ($\mu = 0$) y con coeficientes de restitución perfectamente elásticos ($e = 1$), la energía mecánica Et debe permanecer constante. Las fluctuaciones menores en el CSV exportado corresponderán estrictamente a los errores de truncamiento del punto flotante en C.

Diseño del Simulador en C

El simulador está diseñado bajo una arquitectura modular y estructurada en C, separando las estructuras de datos, la resolución matemática de los eventos físicos mediante métodos y los bucles principales presentes en el main.

1. Estructuras de Datos

El código utiliza dos estructuras principales para organizar la información de cada partícula y del entorno.

El código utiliza dos estructuras principales para organizar la información:

- **Partícula**

  Representa a cada objeto de la simulación, las partículas. Se especifican las siguientes características:

  - **Cinemática:** Posición actual (x,y) y su velocidad (vx, vy)
  - **Inercia y Dimensión:** Masa (m) para los cálculos de inercia e impulso, y radio (r).

```c
typedef struct {
    double x, y;   // Posición actual de la partícula
    double vx, vy; // Velocidad actual de la partícula
    double m;   // Masa de la partícula
    double r;  // Radio de la partícula
} Particula;
```

- **Config**

Representa el estado del sistema y define las variables de entorno del simulador.

- **Dimensiones:** Anchura (ancho) y altura (alto) del plano.
- **Parámetros de ticks:** Cantidad total de iteraciones (ticks) y el paso temporal por cada tick (dt).
- **Fuerzas Externas**: Aceleración de la gravedad (g) y coeficiente de rozamiento (mu).
- **Paredes**: Coeficiente de restitución (e) que define la pérdida de energía en los bordes.

```
typedef struct {
    int n_particulas; // Número total de partículas en la simulación
    int ticks;        // Número total de iteraciones (ticks) a
simular
    double dt;        // Paso de tiempo para cada tick
    double g;  // Aceleración debida a la gravedad
    double e;         // Coeficiente de restitución para colisiones
con paredes (0-1)
    double mu;        // Coeficiente de rozamiento (0-1)
    double ancho;     // Ancho del entorno de simulación
    double alto;      // Alto del entorno de simulación
} Config;
```

2. Métodos y Funciones Físicas

El comportamiento físico se divide en funciones y métodos que reciben punteros (Particula *p) para modificar directamente el espacio de memoria:

- **void actualizar_fisica_basica(Particula *p, Config c)**

Implementa la integración numérica mediante el método de Euler semi-explícito.

- Aplicar gravedad: Adicionar a vy sumando g * dt.

$$vy \leftarrow vy + g*dt$$

- Aplicar amortiguamiento viscoso (fricción): Multiplicar vx y vy por (1 - μ * dt).

$$vy \leftarrow vy * (1- mu * dt)$$

- Actualizar posición x sumando vx * dt.

$$x \leftarrow x + vx * dt$$

- Actualizar posición y sumando vy * dt.

$$y \leftarrow y + vy * dt$$

```c
void actualizar_fisica_basica(Particula *p, Config c) {
    /* Gravedad */
    p->vy += c.g * c.dt;

    /* Rozamiento */
    p->vx *= (1.0 - c.mu * c.dt);
    p->vy *= (1.0 - c.mu * c.dt);

    /* Posición */
    p->x += p->vx * c.dt;
    p->y += p->vy * c.dt;
}
```

- **void resolver_colision_paredes(Particula *p, Config c)**

  Evalúa la posición de la partícula frente a los cuatro límites del recinto. Si detecta penetración, aplica una corrección posicional estática y multiplica la velocidad perpendicular a la pared por el negativo del coeficiente de restitución (-e).

```c
void resolver_colision_paredes(Particula *p, Config c) {
    if (p->x < p->r) {
        p->x = p->r;
        p->vx *= -c.e;
    }
    if (p->x > c.ancho - p->r) {
        p->x = c.ancho - p->r;
        p->vx *= -c.e;
    }
    if (p->y < p->r) {
        p->y = p->r;
        p->vy *= -c.e;
    }
    if (p->y > c.alto - p->r) {
        p->y = c.alto - p->r;
        p->vy *= -c.e;
    }
}
```

- **void resolver_colision_particulas(Particula *p1, Particula *p2)**

  Algoritmo central para la interacción entre dos cuerpos. Calcula la distancia euclidiana entre centros $d = \sqrt{dx^2 + dy^2}$.

- Si d < r_1 + r_2, separa geométricamente los cuerpos para eliminar el solapamiento, distribuyendo el desplazamiento de manera inversamente proporcional a sus masas.
- Calcula el producto punto de la velocidad relativa a lo largo del vector normal de colisión.
- Aplica el intercambio de impulso elástico, modificando las velocidades en función de un factor ponderado por la masa de ambas partículas (Impulso /= 1 / (1/m_1 + 1/m_2)).

```c
void resolver_colision_particulas(Particula *p1, Particula *p2) {
    double dx = p2->x - p1->x;
    double dy = p2->y - p1->y;
    double dist = sqrt(dx * dx + dy * dy);

    if (dist < p1->r + p2->r) {
        /* Separar partículas basándose en la proporción de sus
masas */
        double solapamiento = p1->r + p2->r - dist;
        double nx = dx / dist;
        double ny = dy / dist;

        double masa_total = p1->m + p2->m;
        double ratio1 = p2->m / masa_total;
        double ratio2 = p1->m / masa_total;

        p1->x -= solapamiento * nx * ratio1;
        p1->y -= solapamiento * ny * ratio1;
        p2->x += solapamiento * nx * ratio2;
        p2->y += solapamiento * ny * ratio2;

        /* Intercambio de impulso para masas diferentes */
        double dvx = p2->vx - p1->vx;
        double dvy = p2->vy - p1->vy;
        double dot_product = dvx * nx + dvy * ny;

        if (dot_product < 0) {
            /* Coeficiente de restitución entre partículas (asumimos
1.0 = elástico ideal) */
            double e = 1.0;
```

```
            double impulso = -(1.0 + e) * dot_product;
            impulso /= (1.0 / p1->m + 1.0 / p2->m);

            double ix = impulso * nx;
            double iy = impulso * ny;

            p1->vx -= ix / p1->m;
            p1->vy -= iy / p1->m;
            p2->vx += ix / p2->m;
            p2->vy += iy / p2->m;
        }
    }
}
```

3. Bucles Principales (main)

La parte principal del simulador es un bucle que itera por cada tick, que avanza el tiempo paso a paso. Dentro de ese bucle, ocurren tres sub-bucles en orden:

- **Bucle A (Movimiento):** Recorre todas las partículas una por una para moverlas (usando la física básica) y las hace rebotar contra las paredes si es necesario.
- **Bucle B (Colisiones):** Compara cada partícula con el resto para ver si chocan entre sí. Está diseñado para probar cada pareja de partículas una sola vez por tick.
- **Bucle C (Energía y Extracción):** Vuelve a repasar todas las partículas para sumar la energía total del sistema (cinética y potencial) y manda los datos de ese instante a los archivos para guardarlos.

```
    for (int t = 0; t < conf.ticks; t++) {
        /* BUCLE A: Físicas individuales y paredes */
        for (int i = 0; i < conf.n_particulas; i++) {
            actualizar_fisica_basica(&p[i], conf);
            resolver_colision_paredes(&p[i], conf);
        }
```

```
        /* BUCLE B: Colisiones entre partículas */
        for (int i = 0; i < conf.n_particulas; i++) {
            for (int j = i + 1; j < conf.n_particulas; j++) {
                resolver_colision_particulas(&p[i], &p[j]);
            }
        }

        /* BUCLE C: Extracción de datos y Energía */
        double ec = 0, ep = 0;
        for (int i = 0; i < conf.n_particulas; i++) {
            ec += 0.5 * p[i].m * (p[i].vx * p[i].vx + p[i].vy *
p[i].vy);
            ep += p[i].m * conf.g * (conf.alto - p[i].y);

            fprintf(f_tray, "%d,%d,%.2f,%.2f,%.2f,%.2f,%.2f,%.2f\n",
t, i, p[i].x, p[i].y,
                    p[i].vx, p[i].vy, p[i].m, p[i].r);
        }
        fprintf(f_ener, "%d,%.2f,%.2f,%.2f\n", t, ec, ep, ec + ep);
    }
```

4. Entrada y Salida (Interacción y Archivos)

El simulador interactúa de forma sencilla con el usuario al principio y guarda un registro detallado al final:

- **Entrada:** Al iniciar, un menú en la consola te pide las dimensiones del espacio y la cantidad de partículas. Te da la opción de cargar físicas predeterminadas de forma rápida, o ingresar manualmente la gravedad, rozamiento, etc. Por último, puedes elegir si quieres que las partículas aparezcan aleatoriamente o ingresar sus datos uno por uno.

- **Manejo de Archivos (Salida):** Durante la simulación, el programa exporta los resultados automáticamente a tres archivos CSV:
    - **trayectorias.csv:** Guarda un historial completo. Por cada *tick*, anota dónde estaba cada partícula, a qué velocidad iba, su masa y su radio.
    - **energia.csv:** Registra cómo cambia la energía (cinética, potencial y total) en cada paso. Es ideal para ver cómo el sistema pierde energía por los rebotes y el rozamiento.
    - **benchmark.csv:** Se abre en modo adición (append) para no borrar pruebas anteriores. Guarda cuántas partículas se simularon y cuántos segundos tardó tu computadora en procesarlo, útil para medir el rendimiento del código.

5. Análisis algorítmico

- **Bucle A (Parte 1):** $O(5N) = O(N)$ Cinco operaciones constantes por partícula (2 sumas, 3 multiplicaciones), iteradas N veces.
- **Bucle B (Parte 2):** $O(4N) = O(N)$ Cuatro comparaciones de límites por partícula, iteradas N veces.
- **Bucle B:** $O(N \cdot (N-1)/2) = O((N^2-N)/2) = O(N^2)$ Se elimina el término $-N$ por ser de menor orden, y la constante 1/2 desaparece en notación Big-O.
- **Bucle C:** $O(3N) = O(N)$ Una pasada para calcular energías y otra para escribir el CSV, ambas sobre N partículas.

$O(5N) + O(4N) + O(N \cdot (N-1)/2) + O(3N) = O(11N + (N^2-N)/2) = O(11N + N^2/2 - N/2) = O(N^2/2 + 10.5N) =$ **$O(N^2)$**

Se elimina el término 10.5N por ser de menor orden que N^2, y la constante 1/2 desaparece, quedando $\mathbf{O(N^2)}$ por tick y $\mathbf{O(T \cdot N^2)}$ en total.

Optimización Algorítmica de Colisiones mediante Particionamiento (Grid Hashing)

Para superar la complejidad computacional O($\mathbf{N^2}$) por tick inherente a la comprobación de colisiones por fuerza bruta, se implementó una optimización basada en particionamiento espacial (Grid Hashing). El entorno de simulación se divide en una cuadrícula virtual bidimensional de C celdas uniformes, donde cada celda tiene un lado L = 2*max_r (siendo max_r el radio máximo de las partículas en el sistema).

1. **Funcionamiento Algorítmico y Mapeo Espacial**

El éxito de esta optimización radica en evitar que el algoritmo compare partículas que se encuentran físicamente muy lejos. Para lograrlo, al inicio del bloque de colisiones en cada iteración temporal (tick), el programa ejecuta un proceso de clasificación en dos fases:

- **Fase 1: Mapeo y Construcción de la Cuadrícula (Inicio del ciclo)**

  Antes de evaluar cualquier choque, la estructura espacial se reinicia o "limpia". A continuación, el algoritmo recorre cada partícula y traduce su posición continua (x, y) a una coordenada discreta o índice de cuadrícula. Esto se logra dividiendo la posición de la partícula entre el tamaño de la celda y truncando el valor a un número entero (cx = floor(x/L), cy = floor(y/L)).

  De este modo, la partícula queda asignada instantáneamente en la celda correspondiente. Para organizar esta información de manera rápida sin fragmentar la memoria, el algoritmo

utiliza arreglos paralelos que funcionan como listas enlazadas, agrupando los identificadores de todas las partículas que cayeron en la misma celda. Un detalle arquitectónico crucial es que estos arreglos masivos se declaran en el ámbito global. Esta decisión garantiza una memoria estática, plana y contigua, erradicando el riesgo de desbordamiento de pila (Stack Overflow) y evitando la penalización de latencia que generaría la asignación dinámica de memoria en tiempo de ejecución.

```
#define MX 500005
const unsigned int p1 = 73856093, p2 = 19349663;

...
    for (int t = 0; t < conf.ticks; t++) {
      ...

        memset(head, -1, hash_size * sizeof(int));
        /* B.2 - Llenar la tabla Hash */
        for (int i = 0; i < conf.n_particulas; i++) {
            int cx = (int)(p[i].x / lado_celda);
            int cy = (int)(p[i].y / lado_celda);

            unsigned int h = hash(cx, cy, hash_size);

            next[i] = head[h];
            head[h] = i;
        }
```

- **Fase 2: Evaluación de Vecindarios Locales**

  Una vez que todas las partículas están clasificadas en sus respectivas celdas, el algoritmo procede a buscar colisiones. Sin embargo, ahora una partícula ya no se compara contra el resto del universo, sino únicamente contra las partículas de su misma celda y las de las

celdas directamente adyacentes. Para evitar cálculos duplicados, el algoritmo solo revisa 4 de las 8 direcciones vecinas (derecha, abajo-izquierda, abajo y abajo-derecha).

```
        for (int i = 0; i < conf.n_particulas; i++) {
            int cx = (int)(p[i].x / lado_celda);
            int cy = (int)(p[i].y / lado_celda);

            /* a) Choques dentro de la MISMA celda (se usa j > i para no
repetir) */
            unsigned int h = hash(cx, cy, hash_size);
            int j = head[h];
            while (j != -1) {
                if (j > i) {
                    /* Verificar colisión real de celda (evitar falsos
positivos del Hash) */
                    int cjx = (int)(p[j].x / lado_celda);
                    int cjy = (int)(p[j].y / lado_celda);
                    if (cx == cjx && cy == cjy) {
                        resolver_colision_particulas(&p[i], &p[j]);
                    }
                }
                j = next[j];
            }

            /* b) Choques con celdas VECINAS (4 direcciones) */
            int dx[4] = {1, -1, 0, 1};
            int dy[4] = {0, 1, 1, 1};
            for (int o = 0; o < 4; o++) {
                int nx = cx + dx[o];
                int ny = cy + dy[o];

                unsigned int h_vecina = hash(nx, ny, hash_size);
                int nj = head[h_vecina];

                while (nj != -1) {
                    /* Verificar que pertenecen a la coordenada vecina
exacta */
                    int cnjx = (int)(p[nj].x / lado_celda);
```

```
                int cnjy = (int)(p[nj].y / lado_celda);
                if (nx == cnjx && ny == cnjy) {
                    resolver_colision_particulas(&p[i], &p[nj]);
                }
                nj = next[nj];
            }
        }
    }
```

2. Fundamento Matemático y Probabilístico

Para demostrar matemáticamente la eficiencia de este filtro, modelamos el sistema probabilísticamente. Para una cuadrícula de C celdas, definimos la variable aleatoria:

X_i: Número de partículas cuyo centro geométrico se encuentra en la celda i

Dado que el algoritmo de Hashing Espacial registra a las partículas basándose únicamente en sus coordenadas centrales, la región de aceptación (área favorable) para que una partícula sea asignada a una celda corresponde exactamente al área geométrica de dicha celda. Sabiendo que el lado de cada cuadrícula está definido en función del diámetro máximo de las partículas L = 2r, el área favorable para el registro en una sola celda es:

$$A_{favorable} = L^2 = (2r)^2 = 4r^2$$

Sabiendo que el área total del entorno simulado es la suma del área de las C cuadrículas ($A_{total} = C \times 4r^2$), y asumiendo una distribución espacial aleatoria uniforme, la probabilidad p de que el centro de una partícula arbitraria recaiga sobre una celda específica i está dada por el cociente:

$$p = \frac{A_{favorable}}{A_{total}} = \frac{4r^2}{C \cdot 4r^2} = \frac{1}{C}$$

3. Modelado Estadístico

Dado que la ubicación de cada partícula es independiente y todas tienen la misma probabilidad p de que el centro recaiga en la celda i, la variable aleatoria X_i sigue una distribución Binomial:

$$X_i \sim Binomial(N, p)$$

En simulaciones físicas, el número de partículas N tiende a ser muy grande y la probabilidad p de caer en una celda específica es muy pequeña. Bajo estas condiciones asintóticas, la distribución Binomial se aproxima con alta precisión mediante una distribución de Poisson:

$$X_i \sim Poisson(\lambda)$$

Donde el parámetro λ se define como:

$$\lambda = N \times p = N\left(\frac{1}{C}\right) = \frac{N}{C}$$

Por propiedad de la distribución Poisson, tenemos sobre el valor estimado:

$$E(X_i) = \lambda = \frac{N}{C}$$

En el algoritmo por fuerza bruta, evaluar las colisiones requiere que cada partícula se compare contra las N entidades del sistema. Sin embargo, gracias a este filtro, cada partícula ahora solo necesita verificar colisiones contra las entidades presentes en su propia celda y en 4 celdas vecinas adyacentes. En promedio, la cantidad de comprobaciones que debe realizar cada

partícula se reduce drásticamente a $5E(X_i) = \frac{5N}{C}$ partículas. Al aplicar este límite a las N partículas del simulador, la complejidad computacional de la fase de colisiones desciende a:

$$O(\frac{5N}{C} \times N) = O(\frac{N^2}{C})$$

El caso más relevante para la optimización ocurre cuando el sistema se estabiliza dimensionando la cuadrícula en proporción a la cantidad de partículas.Si se diseña el entorno para que el número total de celdas sea mayor o igual al número de entidades ($C \geq N$), el parámetro λ se acota superiormente por una constante matemática:

$$N \leq C \rightarrow \frac{N}{C} \leq 1 \rightarrow E(X_i) = \lambda \leq 1$$

Al evaluar solo la celda residente y la mitad de sus vecindades adyacentes, el límite de comprobaciones de choques por partícula se vuelve una constante aproximada de $K \approx 5$. Esto optimiza la complejidad algorítmica del subsistema de colisiones, transformando la ineficiente curva cuadrática $O(N^2)$ en una relación lineal $O(\frac{N}{C} \times N) = O(N)$, garantizando que el simulador pueda procesar densidades de partículas masivas sin comprometer los tiempos de ejecución.

4. Gestión de Memoria: Cuadrícula Dispersa y Función Hash

Una implementación ingenua de la cuadrícula requiere reservar memoria para todo el dominio espacial ($O(C) = O(\frac{Ancho \times Alto}{L^2})$). Si el área de simulación es inmensamente grande en comparación con la cantidad de partículas, la matriz se vuelve dispersa y la mayoría de las celdas estaran vacías, lo que genera un desperdicio de memoria y un cuello de botella innecesario al reiniciar la grilla en cada iteración.

Para solucionar esto, se aplica el verdadero concepto de Hashing Espacial. En lugar de mapear el espacio 1:1 en la memoria, se utiliza una Tabla Hash unidimensional alojada en la memoria global, cuyo tamaño es estrictamente proporcional al número de partículas. Dado que, en el peor de los casos probabilísticos, las partículas están aisladas y revisan sus celdas vecinas, la cantidad de "ranuras" de memoria necesarias está acotada por $M \approx 5N$. Para disminuir la probabilidad de colisiones de mapeo, en nuestro modelo utilizaremos $M = 12N$.

Se emplea una función de dispersión (Hash Function) que proyecta las coordenadas discretas bidimensionales (C_x, C_y) hacia el índice unidimensional de la tabla:

$$hash(C_x, C_y) = (C_x . p_1 \oplus C_y . p_2) \bmod M$$

Donde p_1 y p_2 son números primos grandes, nosotros utilizamos p_1= 73856093 y p_2= 19349663) que minimizan las colisiones de hash.

```
const unsigned int p1 = 73856093, p2 = 19349663;

...

      unsigned int hash(int cx, int cy, int hash_size) {
          /* XOR bit a bit de coordenadas multiplicadas por primos, mapeado al
tamaño de la tabla */
          return (unsigned int)((cx * p1) ^ (cy * p2)) % hash_size;
      }
```

Con esta arquitectura global y optimizada, tanto la complejidad temporal (ciclos de CPU) como la espacial (consumo de RAM) quedan completamente desacopladas del tamaño del mundo físico, limitándose estrictamente a O(N), generando una complejidad total de O(T*N).

6. Entrada interactiva del usuario (módulo entrada.c)

Se implementó el módulo entrada.c para que el usuario pueda tener la opción de personalizar los parámetros físicos para las partículas como cantidad de partículas, gravedad, velocidad inicial, masa y radio. Si elige saltarse la configuración, el programa arranca de inmediato con valores aleatorios para dar inicialización al programa.

El módulo de entrada sigue la secuencia de un menú con dos preguntas. La opción manual es pedirle al usuario que escriba con el teclado, o pedirle al programa que lo genere de forma aleatoria dentro de un rango definido. La segunda pregunta, cuando se aplica, es a qué partículas afecta ese valor: a todas por igual, o uno distinto para cada una. La combinación de estas dos decisiones cubre desde experimentos completamente reproducibles, donde el usuario controla cada número. Cuando el alcance es por partícula, el módulo reserva un arreglo dinámico de tamaño N para guardar los valores individuales, mientras que cuando el alcance es global se conserva un único valor que aplica a todo el sistema.

El módulo trabaja con bibliotecas estándar de C: stdio.h para la lectura por consola y la escritura de mensajes, stdlib.h para reservar memoria dinámica y generar valores aleatorios, y string.h para manipular el buffer de entrada y expone un par de funciones públicas hacia el resto del programa donde una que ejecuta el menú completo y otra que libera la memoria reservada al final de la simulación.

Archivos CSV

El simulador en C y los gráficos en Python no se comunican directamente. Entre ambos existe una capa intermedia formada por tres archivos de texto en formato CSV que el simulador

genera automáticamente al ejecutarse. La dinámica reside en que el simulador genera los archivos en un formato bien definido y el script de Python los lee asumiendo ese mismo formato. La generación de los CSV está concentrada en un módulo dedicado que solo necesita las funciones estándar de manejo de archivos en C (fopen, fprintf, fclose).

El primer archivo guarda la trayectoria de cada partícula a lo largo del tiempo. Por cada tick de la simulación se escriben N filas, una por partícula, registrando su posición y velocidad redondeadas a cuatro decimales. Esa precisión es suficiente para reconstruir trayectorias suaves al graficarlas y mantiene el archivo en un tamaño manejable.

El segundo archivo guarda las energías del sistema entero (cinética, potencial y total) escribiendo una única fila por tick con seis decimales de precisión. Usar esta cantidad de decimales permite detectar derivas numéricas pequeñas que serían invisibles con menos resolución, lo que resulta esencial para validar la conservación de energía como prueba de calidad del integrador físico.

El tercer archivo, dedicado al benchmark, se maneja distinto a los otros dos. En lugar de reescribirse en cada corrida, se abre en modo de adición (append), de modo que cada ejecución del simulador agrega una fila nueva con el número de partículas y el tiempo total que tardó. Antes de escribir, el módulo verifica si el archivo ya existía: si es nuevo, escribe primero la fila de encabezados; si ya existía, solo agrega los datos. Esta lógica permite ejecutar el simulador con diferente número de partículas N para tener los resultados listos a graficar.

Tabla 1.

Resumen de los archivos csv generados.

Archivo generado	Columnas registradas	Descripción
trayectorias.csv	tick, particula_id, x, y, vx, vy	Registra la cinemática de cada partícula en cada instante de tiempo.
energia.csv	tick, energia_cinetica, energia_potencial, energia_total	Permite auditar el estado energético y la conservación mecánica del sistema.
benchmark.csv	n_particulas, ticks, tiempo_segundos, particulas_por_segundo	Almacena las métricas de rendimiento y tiempo de ejecución evaluadas.

Nota: Elaboración propia.

Visualizador en Python

El script de visualización es independiente del simulador y se ejecuta después, una vez que los CSV ya están generados. Su única tarea es leer esos archivos y convertir los números en imágenes que cuenten una historia comprensible. Para lograrlo se apoya en cuatro bibliotecas del

ecosistema científico de Python, cada una con un rol específico que se conecta con las demás a través de estructuras de datos compartidas.

Las bibliotecas y su rol

pandas: lectura y manipulación tabular

Pandas se utiliza para leer y manipular los archivos CSV. Cada archivo se carga como un objeto conocido como DataFrame, la ejecución prevé que los datos existan, que tengan adecuado formato, o si se puede acceder al programa. Una vez en memoria, los DataFrames se manipulan con operaciones estándar de pandas: filtrar las filas que corresponden a un tick específico, obtener la lista de identificadores únicos de partícula, ordenar las corridas del benchmark por tamaño antes de graficar, o acceder a una columna por nombre para pasarla como serie temporal a las gráficas.

numpy: operaciones vectoriales

Numpy complementa a pandas en las operaciones que requieren cálculo vectorial puro. Se usa en la generación del histograma bidimensional para el mapa de densidad, donde una sola llamada vectorizada agrupa cientos de miles de coordenadas en una grilla fina sin necesidad de bucles en Python, y la construcción de las mallas de coordenadas necesarias para que las gráficas tipo heatmap puedan renderizarse.

matplotlib.animation

matplotlib es la biblioteca central de visualización. Al inicio del script se aplica una configuración estética global que afecta a todas las figuras producidas como tamaño por defecto,

fuente con cobertura completa de caracteres latinos extendidos para que las tildes y la "ñ" se rendericen correctamente, estilo de cuadrícula con líneas discontinuas y opacidad reducida, ejes superiores y derechos ocultos, fondo blanco. Cada figura individual se construye creando primero un lienzo con uno o más paneles, dibujando sobre ellos las series, anotaciones y leyendas, y guardándola finalmente como PNG. Después de cada guardado la figura se cierra explícitamente para liberar memoria.

Pillow: animación GIF

Pillow entra en escena exclusivamente para la animación. Aunque matplotlib tiene un mecanismo propio para serializar GIFs, en la práctica presenta problemas de robustez en algunos sistemas, especialmente con figuras de fondo oscuro. Por eso el script renderiza los cuadros manualmente, para cada tick muestreado actualiza las posiciones de las partículas en la figura, fuerza el redibujado, extrae el buffer de píxeles en bruto y lo convierte en una imagen de Pillow. Cada imagen se cuantiza a una paleta adaptativa de colores definidos para reducir el peso del archivo final, y al terminar todos los cuadros se ensamblan en un único GIF con duración configurable entre cuadros y reproducción en bucle infinito.

Resultados - entregables visuales

El script genera cuatro entregables que conforman el reporte visual del experimento que son animación en formato GIF, gráfico de energías, heatmap y gráfico de rendimiento.

El primero es una animación GIF que muestra el movimiento de las partículas en el tiempo. Cada una mantiene un color fijo para poder seguirla fácilmente, y se usan solo algunos cuadros (alrededor de 60) para evitar que el archivo sea demasiado pesado.

El segundo es un gráfico de energías, donde se representan la energía cinética, potencial y total. También se incluye una línea de referencia con la energía inicial y una indicación de cuánto varía. Este gráfico permite verificar si la simulación conserva la energía como debería, o si existen pérdidas por efectos como la fricción.

El tercero es un mapa de densidad (heatmap) que muestra en qué zonas del espacio pasan más tiempo las partículas. En lugar de trayectorias, se acumulan posiciones y se representan con intensidades de color, lo que facilita identificar patrones globales, como acumulaciones en ciertas regiones.

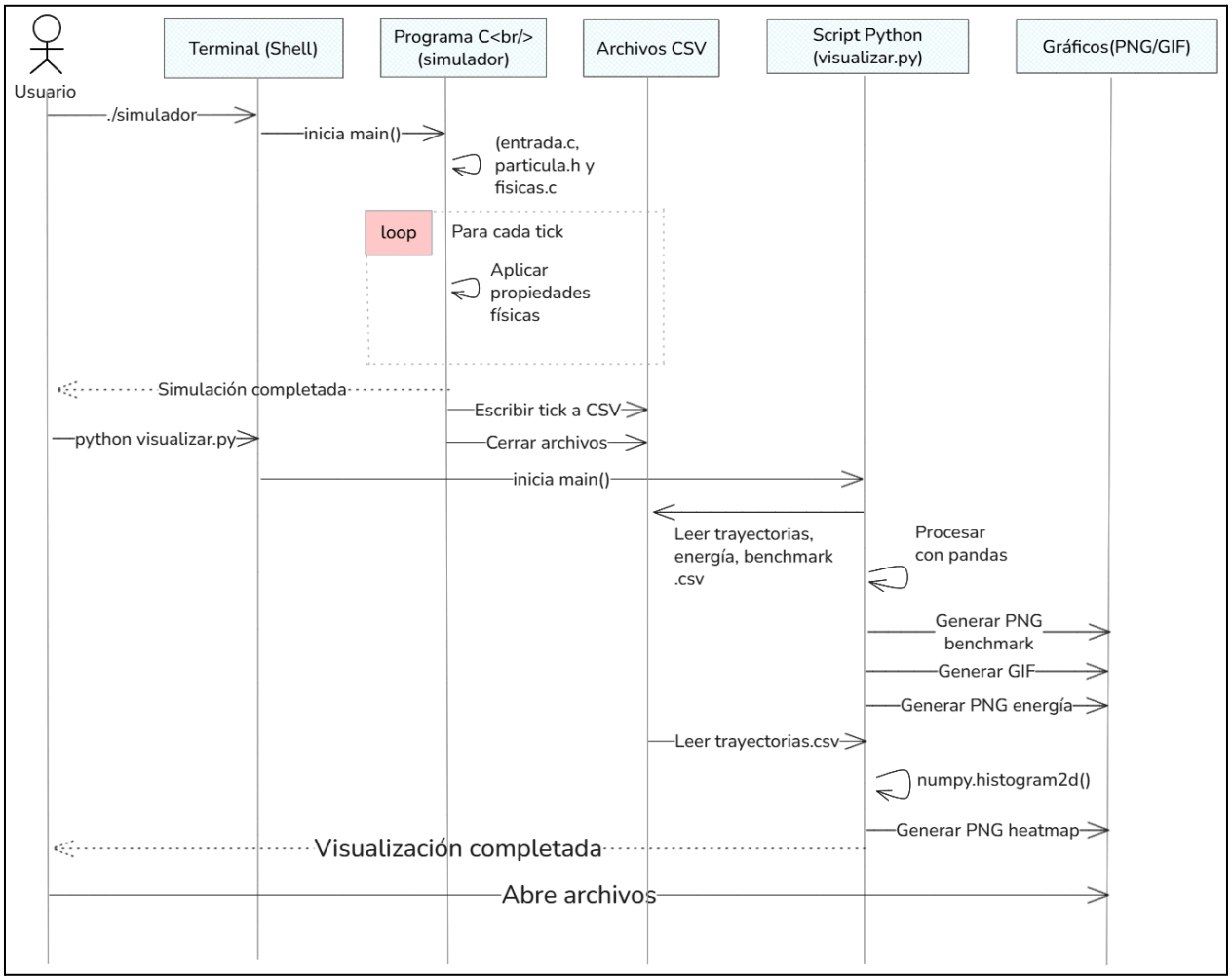
El cuarto es un gráfico de rendimiento, que relaciona el número de partículas con el tiempo de ejecución. Incluye tanto los datos reales como una curva teórica esperada, lo que permite evaluar si el simulador escala correctamente. Además, se muestra una métrica de eficiencia en operaciones por segundo.

Finalmente, el script imprime un resumen en consola indicando qué archivos se generaron, para que el usuario sepa rápidamente los resultados obtenidos.

Diagramas

Figura 1.

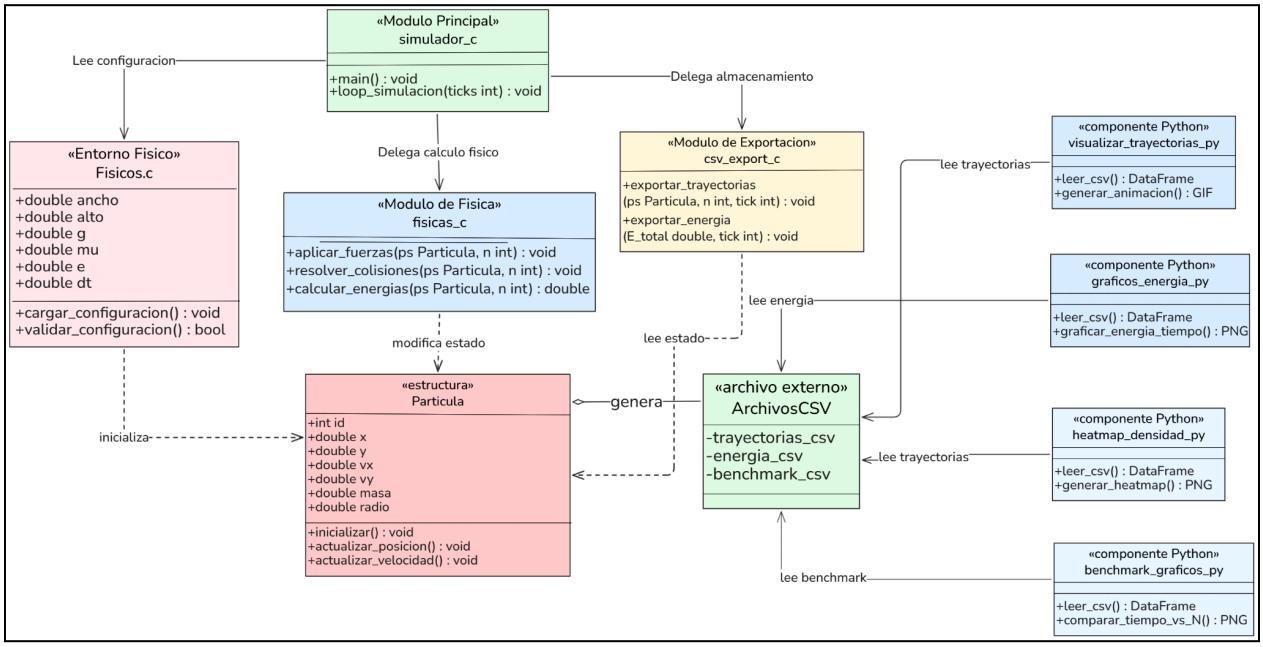
Diagrama de secuencia.



Nota: Elaboración propia con Excalidraw.

Figura 2.

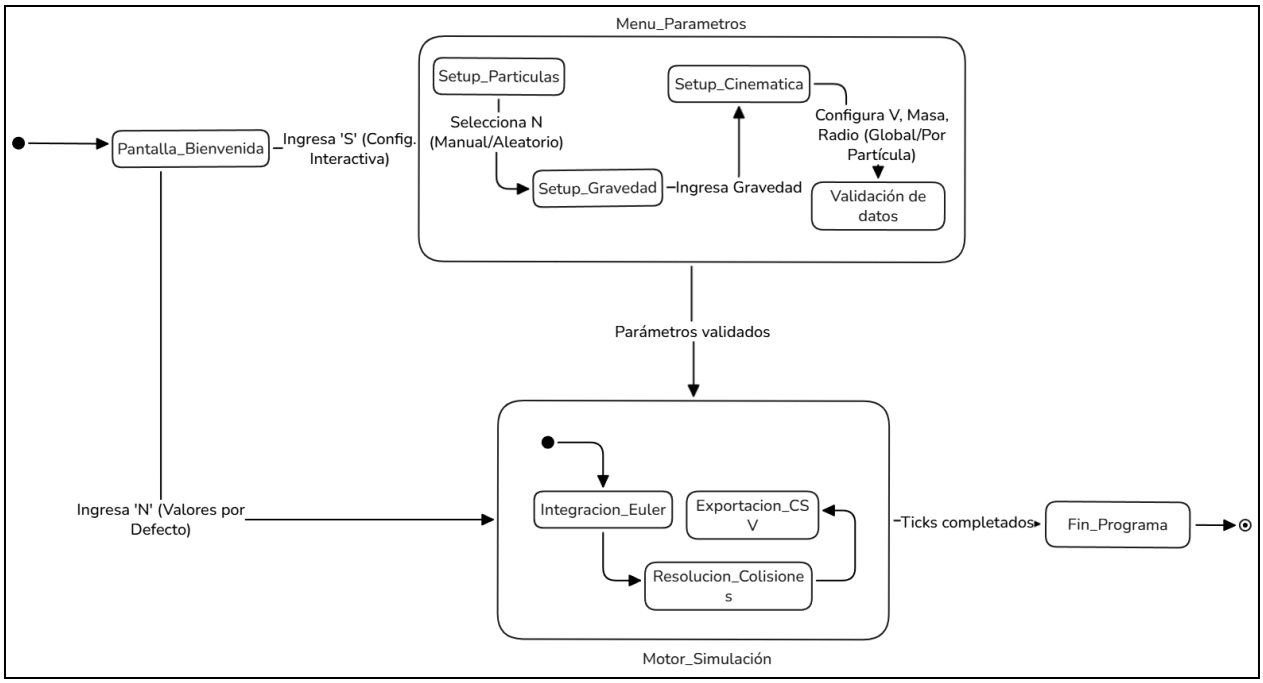
Diagrama de estructuras.



Nota: Elaboración propia con Excalidraw.

Figura 3.

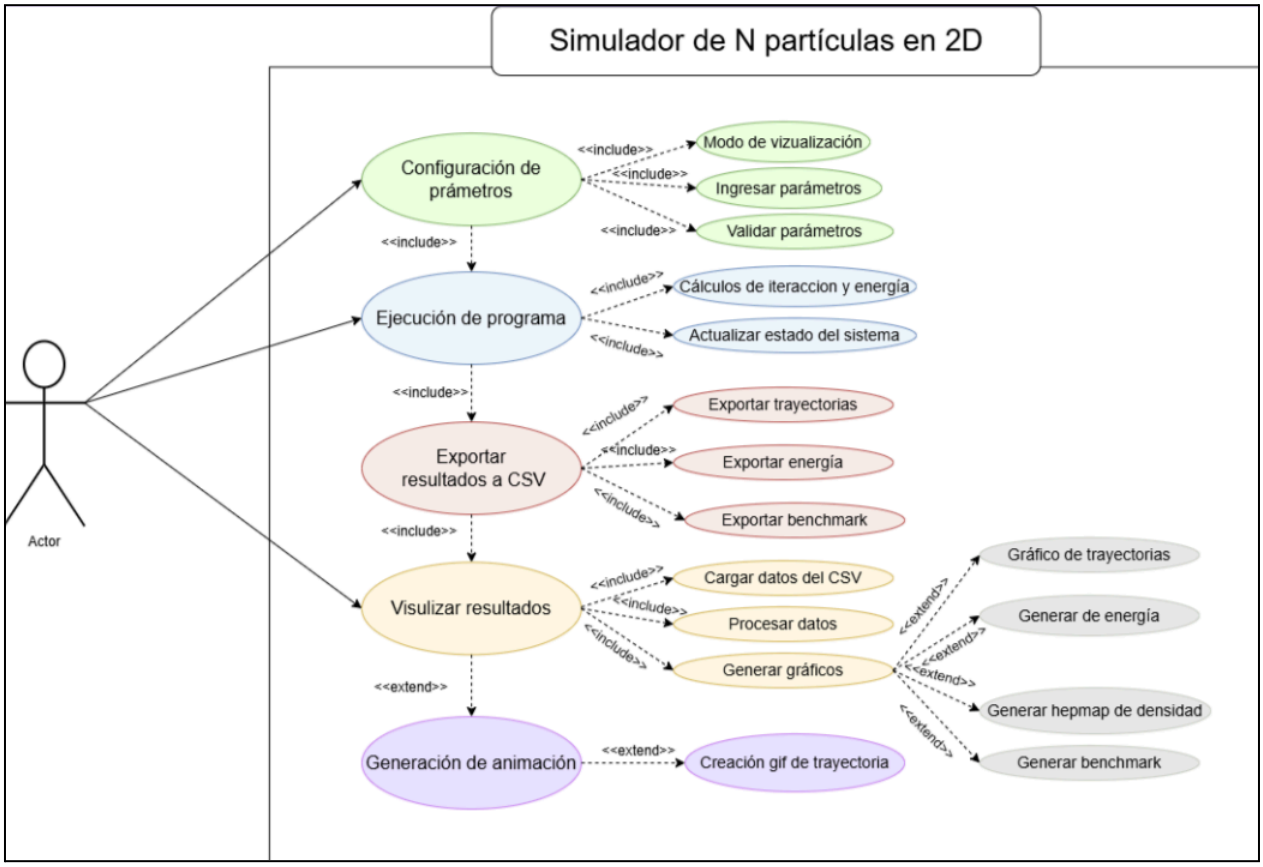
Diagrama de interfaz de usuario basado en texto.



Nota: Elaboración propia con Excalidraw.

Figura 4.

Diagrama de casos de uso.



Nota: Elaboración propia con Draw.io.

Referencias

Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95.

Kernighan, B. W., & Ritchie, D. M. (1991). *El lenguaje de programación C*. Pearson Educación.

Kleppner, D., & Kolenkow, R. (2014). *An introduction to mechanics (2nd ed.)*. Cambridge University Press.

W3Schools.com. (s. f.-a). https://www.w3schools.com/python/numpy/

W3Schools.com. (s. f.-b). https://www.w3schools.com/python/pandas/

Press, W. H. et al. (2007). *Numerical Recipes in C: The Art of Scientific Computing* (3rd Ed.). Cambridge University Press.

Teschner, M. et al. (2003). *Optimized Spatial Hashing for Collision Detection of Deformable Objects*. Proceedings of VMV 2003.